

Cryptocurrencies Report

Lukas Schatzer, Konstantin Unterweger

October 2025

1 Exercise Instructions

For each scenario, briefly describe the rationale behind your solution and include a small code snippet that shows the relevant part of your (or the skeleton's) implementation. You don't have to prove anything formally, only identify which parts of the code capture the behaviour described by the following scenarios.

2 Scenario descriptions

As an introduction, we will briefly explain how the node listens for incoming TCP connections, as it is important to understand the scenario descriptions that follow. The node, written in Python, listens for incoming TCP connections on the specified port using the Python `asyncio` module. It has a built-in option to start a listener:

Listing 1: Python asyncio TCP listener

```
async def listen():
    server = await asyncio.start_server(
        handle_connection, LISTEN_CFG['address'],
        LISTEN_CFG['port'],
        limit=const.RECV_BUFFER_LIMIT
    )
    async with server:
        await server.serve_forever()
```

`asyncio.start_server()` initializes the server. `server.serve_forever()` starts listening for incoming connections. Whenever a client connects, the connection and all related logic are handled in `handle_connection`, which is called with a read stream for receiving messages and a write stream for sending messages.

Messages can be received using `await reader.readline()`, which is a blocking operation. It reads all bytes from the read stream until it encounters a `\n`, then the message is returned.

Sending messages works similarly; we must be careful that our message is canonical JSON, that we encode it to UTF-8, and end it with a `\n`:

Listing 2: Sending messages

```
msg_bytes = json_string.encode()
writer.write(msg_bytes)
writer.write(b'\n')
await writer.drain()
```

`await writer.drain()` blocks until the messages written with `writer.write()` are sent.

2.1 Grader should receive a valid hello message on connecting

Immediately after a client connects, the server initiates the handshake by sending a hello message according to the specifications:

Listing 3: Kerma hello message

```
await write_msg(writer, mk_hello_msg())

def mk_hello_msg():
    return {
        'type': 'hello',
        'version': '0.10.0',
        'agent': 'MyTresure'
    }
```

2.2 The hello message should be followed by a getpeers message

Right after the hello message, the client sends a getpeers message to complete the handshake:

Listing 4: Kerma getpeers message

```
await write_msg(writer, mk_getpeers_msg())

def mk_getpeers_msg():
    return {'type': 'getpeers'}
```

2.3 Grader should be able to disconnect, then connect to your node again.

The initial handshake, as well as the subsequent read loop, is enclosed in a try-except block. If the client disconnects, `await reader.readline()` returns an empty bytes object. This will not pass the `parse_msg` function, and therefore an exception is raised:

Listing 5: Kerma parse_msg function

```
# parses a message as json. returns decoded message
def parse_msg(msg_str):
    try:
        msg = json.loads(msg_str)
    except Exception as e:
        raise ErrorInvalidFormat('Invalid JSON')
    if not isinstance(msg, dict):
        raise ErrorInvalidFormat('Received message not a
            dictionary!')
    if not 'type' in msg:
        raise ErrorInvalidFormat('Key \'type\' not set in
            message!')
    if not isinstance(msg['type'], str):
        raise ErrorInvalidFormat('Key \'type\' is not a
            string!')
    return msg
```

The exception is caught in the except block, and the connection is removed from the set of open connections as well as from the peers database:

Listing 6: Kerma error handling

```
try:
    msg = await reader.readline()
    parse_msg(msg)
    # more code
except:
    print('Closing connection with {}'.format(peer))
    writer.close()
    peer_db.remove_peer(conn)
    del_connection(conn)
```

Afterward, the client can reconnect to our node again. The process is the same as before

2.4 If Grader sends a getpeers message, it must receive a valid peers message

Upon receiving a `getpeers` message, the server responds with a list of peers according to the specifications. This list can contain at most 30 peers and may also be empty.

Listing 7: Kerma peers message

```
if msg_type == 'getpeers':
    await write_msg(writer, mk_peers_msg())

def mk_peers_msg():
    pl = [f'{peer}' for peer in PEERS]
```

```

if len(pl) > 30:
    pl = random.sample(pl, 30)
return {'type': 'peers', 'peers': pl}

```

The `__str__` method of python is overridden to return a string of the form `HOSTNAME:PORT`.

2.5 If Grader sends "type":ge, waits for 0.1 second, then sends tpeers", your node should reply with a valid peers message

This works because, as explained above, `await reader.readline()` keeps the received parts of the message in the buffer and returns them only when encountering a `\n`.

2.6 If Grader sends any message before sending hello, your node should send an error message with name set to `INVALID_HANDSHAKE` and then disconnect

The server expects a valid hello message as the client's initial message; otherwise, the error message with the name `INVALID_HANDSHAKE` is sent, and the connection is closed. The following code shows how the validation of the hello message works:

Listing 8: Kerma validate hello message

```

try:
    firstmsg_str = await reader.readline()
    firstmsg = parse_msg(firstmsg_str)
    validate_hello_msg(firstmsg)
    # More code
except Exception as e:
    try:
        await write_error_msg('INVALID_HANDSHAKE', e.message)
    except:
        pass
    # Remove the connection as seen before

def validate_hello_msg(msg_dict):
    if msg_dict['type'] != 'hello':
        raise ErrorInvalidHandshake('Message type is not \'hello\'!')
    try:
        if 'version' not in msg_dict:
            raise ErrorInvalidFormat('Message malformed: version is missing!')
        version = msg_dict['version']
        if not isinstance(version, str):

```

```

        raise ErrorInvalidFormat(
            'Message malformed: version is not a string!'
        )
    if not re.compile('0\\.10\\.\\d').fullmatch(version):
        raise ErrorInvalidFormat(
            'Version invalid')
    validate_allowed_keys(msg_dict, ['type', 'version',
        'agent'], 'hello')
except ErrorInvalidFormat as e:
    raise e
except Exception as e:
    raise ErrorInvalidFormat('Message malformed: {}'.
        format(str(e)))

```

2.7 If Grader sends an invalid message, your node should send an error message with the correct name

As described above, the received messages are validated. In case of invalid messages, an appropriate error message is sent, and the connection is closed.

2.8 If Grader sends a set of peers in a valid peers message, disconnects, reconnects and sends a getpeers message, it must receive a peers message containing (at least) the peers sent in the first message

When receiving a peers message, the server parses the peers in the message and adds them to the peers database. The peers database is a CSV file that stores the known peers while running and even if the server is restarted. Therefore, after the client reconnects, the getpeers message will contain the previously sent peers. Of course, this works only if the server does not already know many peers; otherwise, it may happen that the previously sent peers are not included in the peers message, because a maximum of 30 peers is sent in that message.

Listing 9: Kerma handle peers message

```

def handle_peers_msg(msg_dict):
    for p in msg_dict['peers']:
        peer_parts = p.rsplit(':', 1)
        host_str, port_str = peer_parts
        port = int(port_str, 10)
        peer = Peer(host_str, port)
        add_peer(peer)

```

2.9 Grader should be able to create two connections to your node simultaneously

When connecting to the server, the client typically uses an ephemeral port assigned by the operating system for the communication. Therefore, multiple connections are not a problem, as each connection can be distinguished by the port used. All of this is handled internally by `asyncio`.